Audited by CRED SHIELDS

CredShields

# Smart Contract Audit

March 16, 2026 • CONFIDENTIAL

## Description

This document details the process and result of the Smart Contract audit performed by CredShields Technologies PTE. LTD. on behalf of Fission Labs between March 9, 2026, and March 11, 2026. A retest was performed on March 15, 2025.


## Author

Shashank (Co-founder, CredShields) shashank@CredShields.com

## Reviewers

Aditya Dixit (Research Team Lead), Shreyas Koli(Auditor), Naman Jain (Auditor), Sanket Salavi (Auditor), Prasad Kuri (Auditor), Neel Shah (Auditor)


## Prepared for

Fission Labs

AICPA SOC | SCS Advocates OWASP OWASP Smart Contract Top 10 Security Risks & Vulnerabilities

# Table of Contents


**1. Executive Summary ——————————-** **3**

State of Security 4

**2. The Methodology ——————————-** **5**

2.1 Preparation Phase 5

2.1.1 Scope 5

2.1.2 Documentation 5

2.1.3 Audit Goals 6

2.2 Retesting Phase 6

2.3 Vulnerability classification and severity 6

2.4 CredShields staff 8

**3. Findings Summary ——————————-** **9**

3.1 Findings Overview 9

3.1.1 Vulnerability Summary 9

**5. Bug Reports ——————————** **10**

Bug ID #L001 [Fixed] 10

Missing Zero-Address Validation in initialize 10

Bug ID #L002 [Fixed] 12

Permit2 Front-Running / Griefing DoS 12

Bug ID #I001 [Acknowledged] 14

Redemption Identifier Reuse After Deletion 14

Bug ID #I002 [Acknowledged] 16

Provider-Controlled redemptionId Allows ID Space Monopolization 16

Bug ID #I003 [Fixed] 18

Checks-Effects-Interactions (CEI) Pattern Violation in Redemption Lifecycle 18

Bug ID #I004 [Acknowledged] 20

Missing Storage Gap in Upgradeable Contract 20

Bug ID #G001 [Acknowledged] 21

Public constants can be private 21

Bug ID #G002 [Fixed] 22

Gas Optimization in Require/Revert Statements 22

**6. The Disclosure ——————————-** **23**

# 1. Executive Summary ----------

Fission Labs engaged CredShields to perform a smart contract audit from March 9, 2026, to March 11, 2026. During this timeframe, 8 vulnerabilities were identified. **A retest was performed on March 15, 2025, and all the bugs have been addressed.**

During the audit, 0 vulnerabilities were found with a severity rating of either High or Critical. These vulnerabilities represent the greatest immediate risk to "Fission Labs" and should be prioritized for remediation, and fortunately, none were found.

The table below shows the in-scope assets and a breakdown of findings by severity per asset. Section 2.3 contains more information on how severity is calculated.

| Assets in Scope | Critical | High | Medium | Low | info | Gas | Σ |
|---|---|---|---|---|---|---|---|
| Fission Redemption Manager | 0 | 0 | 0 | 2 | 4 | 2 | **8** |

*Table: Vulnerabilities Per Asset in Scope*

The CredShields team conducted the security audit to focus on identifying vulnerabilities in Fission Redemption Manager's scope during the testing window while abiding by the policies set forth by Fission Labs's team.

## State of Security

To maintain a robust security posture, it is essential to continuously review and improve upon current security processes. Utilizing CredShields' continuous audit feature allows both Fission Labs's internal security and development teams to not only identify specific vulnerabilities but also gain a deeper understanding of the current security threat landscape.

To ensure that vulnerabilities are not introduced when new features are added, or code is refactored, we recommend conducting regular security assessments. Additionally, by analyzing the root cause of resolved vulnerabilities, the internal teams at Fission Labs can implement both manual and automated procedures to eliminate entire classes of vulnerabilities in the future. By taking a proactive approach, Fission Labs can future-proof its security posture and protect its assets.

# 2. The Methodology ----------

Fission Labs engaged CredShields to perform a Fission Redemption Manager audit. The following sections cover how the engagement was put together and executed.

## 2.1 Preparation Phase

The CredShields team meticulously reviewed all provided documents and comments in the smart contract code to gain a thorough understanding of the contract's features and functionalities. They meticulously examined all functions and created a mind map to systematically identify potential security vulnerabilities, prioritizing those that were more critical and business-sensitive for the refactored code. To confirm their findings, the team deployed a self-hosted version of the smart contract and performed verifications and validations during the audit phase.

A testing window from March 9, 2026, to March 11, 2026, was agreed upon during the preparation phase.

### 2.1.1 Scope

During the preparation phase, the following scope for the engagement was agreed upon:

| IN SCOPE ASSETS |
| --- |
| • **Audited Scope:** https://github.com/fission-xyz/blockchain/blob/1bb55f6b7d51dd6de3a7203e5b23806f380bc7ef/contracts/fission-redemption/FissionRedemptionManager.sol<br>• **Retested Scope:** https://github.com/fission-xyz/blockchain/blob/2ce9c1d22e0fa666fa1df548d6603a36e43d29a9/contracts/fission-redemption/FissionRedemptionManager.sol |

### 2.1.2 Documentation

Documentation was not required as the code was self-sufficient for understanding the project.

### 2.1.3 Audit Goals

CredShields employs a combination of in-house tools and thorough manual review processes to deliver comprehensive smart contract security audits. The majority of the audit involves manual inspection of the contract's source code, guided by OWASP's Smart Contract Security Weakness Enumeration (SCWE) framework and an extended, self-developed checklist built from industry best practices. The team focuses on deeply understanding the contract's core logic, designing targeted test cases, and assessing business logic for potential vulnerabilities across OWASP's identified weakness classes.

CredShields aligns its auditing methodology with the OWASP Smart Contract Security projects, including the Smart Contract Security Verification Standard (SCSVS), the Smart Contract Weakness Enumeration (SCWE), and the Smart Contract Secure Testing Guide (SCSTG). These frameworks, actively contributed to and co-developed by the CredShields team, aim to bring consistency, clarity, and depth to smart contract security assessments. By adhering to these OWASP standards, we ensure that each audit is performed against a transparent, community-driven, and technically robust baseline. This approach enables us to deliver structured, high-quality audits that address both common and complex smart contract vulnerabilities systematically.

## 2.2 Retesting Phase

Fission Labs is actively partnering with CredShields to validate the remediations implemented towards the discovered vulnerabilities.

## 2.3 Vulnerability classification and severity

CredShields follows OWASP's Risk Rating Methodology to determine the risk associated with discovered vulnerabilities. This approach considers two factors - Likelihood and Impact - which are evaluated with three possible values - **Low**, **Medium**, and **High**, based on factors such as Threat

agents, Vulnerability factors, and Technical and Business Impacts. The overall severity of the risk is calculated by combining the likelihood and impact estimates.

| Overall Risk Severity | | | | |
|---|---|---|---|---|
| Impact | HIGH | Medium | High | Critical |
| | MEDIUM | Low | Medium | High |
| | LOW | None | Low | Medium |
| | | LOW | MEDIUM | HIGH |
| Likelihood | | | | |

Overall, the categories can be defined as described below -

1. **Informational**

   We prioritize technical excellence and pay attention to detail in our coding practices. Our guidelines, standards, and best practices help ensure software stability and reliability. Informational vulnerabilities are opportunities for improvement and do not pose a direct risk to the contract. Code maintainers should use their own judgment on whether to address them.

2. **Low**

   Low-risk vulnerabilities are those that either have a small impact or can't be exploited repeatedly or those the client considers insignificant based on their specific business circumstances.

3. **Medium**

   Medium-severity vulnerabilities are those caused by weak or flawed logic in the code and can lead to exfiltration or modification of private user information. These vulnerabilities

can harm the client's reputation under certain conditions and should be fixed within a specified timeframe.

4. **High**

   High-severity vulnerabilities pose a significant risk to the Smart Contract and the organization. They can result in the loss of funds for some users, may or may not require specific conditions, and are more complex to exploit. These vulnerabilities can harm the client's reputation and should be fixed immediately.

5. **Critical**

   Critical issues are directly exploitable bugs or security vulnerabilities that do not require specific conditions. They often result in the loss of funds and Ether from Smart Contracts or users and put sensitive user information at risk of compromise or modification. The client's reputation and financial stability will be severely impacted if these issues are not addressed immediately.

6. **Gas**

   To address the risk and volatility of smart contracts and the use of gas as a method of payment, CredShields has introduced a "Gas" severity category. This category deals with optimizing code and refactoring to conserve gas.

## 2.4 CredShields staff

The following individual at CredShields managed this engagement and produced this report:

- Shashank, Co-founder CredShields shashank@CredShields.com

Please feel free to contact this individual with any questions or concerns you have about the engagement or this document.

# 3. Findings Summary ----------

This chapter contains the results of the security assessment. Findings are sorted by their severity and grouped by asset and OWASP SCWE classification. Each asset section includes a summary highlighting the key risks and observations. The table in the executive summary presents the total number of identified security vulnerabilities per asset, categorized by risk severity based on the OWASP Smart Contract Security Weakness Enumeration framework.

## 3.1 Findings Overview

### 3.1.1 Vulnerability Summary

During the security assessment, 8 security vulnerabilities were identified in the asset.

| Vulnerability Title | Severity | Vulnerability Type | Status |
|---|---|---|---|
| Missing Zero-Address Validation in initialize | Low | SeverityLow | Fixed |
| Permit2 Front-Running / Griefing DoS | Low | Front-running | Fixed |
| Redemption Identifier Reuse After Deletion | Informational | Business Logic (SC03-LogicErrors) | Acknowledged |
| Provider-Controlled redemptionId Allows ID Space Monopolization | Informational | Business Logic (SC03-LogicErrors) | Acknowledged |
| Checks-Effects-Interactions (CEI) Pattern Violation in Redemption Lifecycle | Informational | Reentrancy Attacks (SCWE-046) | Fixed |
| Missing Storage Gap in Upgradeable Contract | Informational | Upgradeability Best Practice | Acknowledged |
| Public constants can be private | Gas | Gas Optimization (SCWE-082) | Acknowledged |
| Gas Optimization in Require/Revert Statements | Gas | Gas Optimization (SCWE-082) | Fixed |

*Table: Findings and Remediations*

# 5. Bug Reports ----------

Bug ID #L001 [Fixed]

## Missing Zero-Address Validation in initialize

**Vulnerability Type**
Lack of Input Validation (SC04-Lack Of Input Validation)

**Severity**
Low

**Description**
The initialize() function does not validate that critical configuration parameters are non-zero addresses. The parameters initParams.defaultAdmin, initParams.settlementTokenAddress, and initParams.permit2 are directly assigned without checks.

Because initialize uses the initializer modifier, it can only be executed once. If any of these parameters are mistakenly set to the zero address during deployment, the contract may become partially or fully unusable.
Examples of misconfiguration include:

- defaultAdmin = address(0): No account would hold DEFAULT_ADMIN_ROLE, preventing role management or upgrades through the role system.
- settlementTokenAddress = address(0): Calls interacting with the settlement token would revert when attempting token transfers.
- permit2 = address(0): Calls to fulfillRedemptionWithPermit2 would revert when attempting to interact with the Permit2 contract.

While this scenario requires a deployment mistake and cannot be triggered by an attacker, the lack of validation increases the risk of accidental misconfiguration.

**Affected Code**

- FissionRedemptionManager.sol#L49-L59

**Impacts**
If incorrect parameters are provided during initialization, the contract may become difficult or

impossible to operate correctly. This could require redeploying the system or performing an upgrade to correct the configuration.

**Remediation**

Add explicit validation checks to ensure that critical initialization parameters are not the zero address before assigning them.

**Retest**

Zero address validations have been added.

Bug ID #L002 [Fixed]

## Permit2 Front-Running / Griefing DoS

**Vulnerability Type**
Front-running

**Severity**
Low

**Description**
The fulfillRedemptionWithPermit2 unconditionally calls permit2.permit with no error handling:

```
permit2.permit(msg.sender, permitSingleAllowanceData, signature);
```

When a user broadcasts this transaction, the full permit signature and calldata are publicly visible in the mempool before the transaction mines. A Permit2 permit signature is a standalone, replayable authorization — anyone can extract it and submit it directly to the Permit2 contract in a separate transaction. Once the nonce is consumed this way, the victim's original transaction reverts on the permit call, because the same nonce cannot be spent twice.

**Affected Code**

- FissionRedemptionManager.sol#L130-L167

**Proof of Concept**

- Redeemer signs a Permit2 PermitSingle for tokenAmount and broadcasts fulfillRedemptionWithPermit2.
- A MEV bot observes the pending transaction and extracts permitSingleAllowanceData and signature from the calldata.
- The bot calls permit2.permit(victim, permitSingleAllowanceData, signature) directly, consuming the nonce.
- The redeemer's original transaction mines and reaches the affected line. The permit2.permit call reverts because the nonce is already spent.
- The redeemer's fulfillment fails. The redemption remains open, the redeemer's redeemToken was never transferred, and the provider's settlementAmount stays locked in the contract.

- The redeemer must sign a fresh permit with a new nonce and retry — which is front-runnable again indefinitely.

Note that this does not require a sophisticated attacker. Any node operator, searcher, or even a competing redeemer (if the redemption ID were misconfigured) can execute this with a simple mempool listener.

**Impacts**

Any redeemer using the Permit2 fulfillment path can be griefed repeatedly at near-zero cost to the attacker. The fulfillment is blocked, the redemption window ticks down, and if the attack is sustained until expiresTime elapses, the provider reclaims their settlement via closeExpiredRedemption and the redeemer receives nothing despite having a valid, signed permit. The standard fulfillRedemption path (direct ERC-20 approval) is unaffected, but the existence of this vector effectively makes fulfillRedemptionWithPermit2 unreliable as a user-facing feature.

**Remediation**

Wrap permit2.permit in a try-catch. If the permit call reverts, (whether because it was front-run or already applied) execution should continue, relying on the allowance that already exists on the Permit2 contract.

**Retest**

The contract now wraps permit2.permit(...) in a try/catch block with an empty catch, allowing execution to continue if the permit was already consumed. The function will still revert at the permit2.transferFrom call if the allowance was never legitimately set, preserving correctness.

Bug ID #I001 [Acknowledged]

## Redemption Identifier Reuse After Deletion

**Vulnerability Type**
Business Logic (SC03-LogicErrors)

**Severity**
Informational

**Description**

The contract deletes redemption entries after they are fulfilled or closed using:

```
delete _redemptions[redemptionId];
```

In Solidity, deleting a mapping entry resets its values to the default state. The contract determines whether a redemption ID is unused by checking:

```
_redemptions[redemptionId].terms.expiresTime == 0
```

After deletion, expiresTime becomes 0, meaning the same redemptionId can be reused for a new redemption. As a result, previously used identifiers may represent multiple redemption lifecycles over time.

This behavior does not introduce an on-chain vulnerability because the contract treats each lifecycle independently and emits events (RedemptionReady, RedemptionFulfilled, RedemptionExpired) that allow full lifecycle tracking. However, since redemption identifiers are externally supplied rather than generated through a strictly increasing counter, reuse may cause ambiguity for off-chain systems that assume identifiers are globally unique.

**Affected Code**

- FissionRedemptionManager.sol#L66
- FissionRedemptionManager.sol#L92

**Impacts**

If off-chain services such as indexers, analytics tools, or routing systems rely solely on redemptionId as a globally unique identifier, reused identifiers may cause ambiguity when reconstructing historical redemption data. This could lead to incorrect analytics, duplicated records, or difficulty correlating events across redemption lifecycles.

**Remediation**

If global uniqueness of redemption identifiers is required, consider preventing reuse by maintaining a record of previously used IDs or by generating identifiers internally using a monotonic counter. Alternatively, ensure off-chain services track redemption lifecycles using emitted events rather than assuming identifier uniqueness.

**Retest**

Client's comment: Noted, and did not fix. We will use a UUID generated from out backend and stored in a SQL DB, which will make the ID globally unique. So we will operationally prevent this issue.

Bug ID #I002 [Acknowledged]

## Provider-Controlled redemptionId Allows ID Space Monopolization

**Vulnerability Type**
Business Logic (SC03-LogicErrors)

**Severity**
Informational

**Description**
The createRedemption function allows providers to supply an arbitrary redemptionId when creating a new redemption entry. The contract only validates that the ID has not been previously used by checking whether the stored entry is empty (e.g., expiresTime == 0).

Because the identifier is entirely user-controlled, any account with PROVIDER_ROLE can preemptively create redemptions using IDs that other providers intend to use. Since the contract prevents overwriting existing entries, once an ID is claimed it becomes unavailable until the redemption is fulfilled or expired and deleted.

If multiple providers exist in the system, a malicious or compromised provider can repeatedly front-run other providers' createRedemption transactions by submitting the same redemptionId. This allows the attacker to monopolize the identifier space and block legitimate redemptions from being created with those IDs.

While the protocol itself remains functional, such behavior can cause off-chain routing failures or coordination breakdowns if external services rely on predetermined redemption identifiers.

**Affected Code**

- FissionRedemptionManager.sol#L62-L78

**Impacts**
If multiple providers hold PROVIDER_ROLE, a malicious provider can monopolize the identifier namespace by preemptively claiming IDs that other providers intend to use. This can lead to:

- Failed redemption creation transactions for legitimate providers
- Off-chain coordination failures if routing systems rely on predetermined redemption IDs
- Operational denial-of-service between providers sharing the same protocol infrastructure

The attack does not result in direct fund loss but can disrupt redemption creation and coordination within the system.

**Remediation**

Remove the externally supplied redemptionId parameter and instead generate identifiers internally. A common approach is to maintain an auto-incrementing counter and assign identifiers sequentially. It also prevents identifier reuse across redemption lifecycles, simplifying off-chain indexing.

This ensures identifiers cannot be monopolized by external actors and prevents front-running between providers.

**Retest**

Client's comment: Noted, and did not fix. Fission will be the only one with `PROVIDER_ROLE`, so we will operationally prevent this issue.

Bug ID #I003 [ Fixed ]

## Checks-Effects-Interactions (CEI) Pattern Violation in Redemption Lifecycle

**Vulnerability Type**
Reentrancy Attacks (SCWE-046)

**Severity**
Informational

**Description**
In fulfillRedemption, fulfillRedemptionWithPermit2, and closeExpiredRedemption, the contract performs external token transfers before deleting the corresponding redemption entry from the _redemptions mapping.

Because the internal state is cleared only after the external interaction, the contract temporarily exposes stale state during execution. If the transferred token supports callbacks (e.g., ERC777 hooks or malicious token implementations), a callback could query the public view function getRedemptionTerms(redemptionId) while the redemption entry still exists.

This may cause the function to return the redemption as valid even though it is already being fulfilled or closed. However, the contract uses the nonReentrant modifier, which prevents state-changing reentrancy, limiting this issue to temporary read-only state exposure.

**Affected Code**
The vulnerability occurs in the following functions:

- FissionRedemptionManager.sol#L108-L128
- FissionRedemptionManager.sol#L130-L167
- FissionRedemptionManager.sol#L81-L94

**Impacts**
External callbacks could observe temporarily stale redemption state through getRedemptionTerms. While this does not affect the contract's internal security, integrations relying on this view function during execution may observe inconsistent data.

**Remediation**

Follow the Checks-Effects-Interactions pattern by updating internal state before performing external calls.

**Retest**

All three affected functions now perform "delete _redemptions[redemptionId]" before any external token transfer calls.

Bug ID #I004 [Acknowledged]

## Missing Storage Gap in Upgradeable Contract

**Vulnerability Type**
Upgradeability Best Practice

**Severity**
Informational

**Description**
The contract inherits from several OpenZeppelin upgradeable base contracts but does not define a storage gap (__gap) for its own storage variables. Storage gaps are sometimes included to reserve space for future variables and simplify storage layout management in upgradeable contracts.

However, the absence of a storage gap does not introduce an immediate vulnerability because new variables added in future upgrades will be appended after the existing storage layout. The current implementation remains upgrade-safe as long as developers avoid modifying or reordering existing storage variables.

**Affected Code**

- FissionRedemptionManager.sol#L21

**Impacts**
No direct security impact. The absence of a storage gap may slightly reduce flexibility when modifying storage layout in future upgrades but does not cause storage collisions under normal upgrade practices.

**Remediation**
Optionally reserve storage slots for future variables.

**Retest**
Client's comment: Noted and did not fix.

Bug ID #G001 [Acknowledged]

## Public constants can be private

**Vulnerability Type**
Gas Optimization (SCWE-082)

**Severity**
Gas

**Description**
Public constant variables cost more gas because the EVM automatically creates getter functions for them and adds entries to the method ID table. The values can be read from the source code instead.

**Affected Code**

- FissionRedemptionManager.sol#L31

**Impacts**
Public constants are more costly due to the default getter functions created for them, increasing the overall gas cost.

**Remediation**
If reading the values for the constants is not necessary, consider changing the public visibility to private.

**Retest**
Client's Comment: Noted and did not fix. We prefer to be able to check those settings with get functions vs contract code reads after deployment.

Bug ID #G002 [Fixed]

## Gas Optimization in Require/Revert Statements

**Vulnerability Type**
Gas Optimization (SCWE-082)

**Severity**
Gas

**Description**
The **require/revert** statement takes an input string to show errors if the validation fails.
The strings inside these functions that are longer than **32 bytes** require at least one additional MSTORE, along with additional overhead for computing memory offset and other parameters. For this purpose, having strings lesser than 32 bytes saves a significant amount of gas.

**Affected Code**

- FissionRedemptionManager.sol#L117-L121

**Impacts**
Having longer require/revert strings than **32 bytes** cost a significant amount of gas.

**Remediation**
It is recommended to shorten the strings passed inside **require/revert** statements to fit under **32 bytes**. This will decrease the gas usage at the time of deployment and at runtime when the validation condition is met.

**Retest**
This is fixed.

# 6. The Disclosure ----------

The Reports provided by CredShields are not an endorsement or condemnation of any specific project or team and do not guarantee the security of any specific project. The contents of this report are not intended to be used to make decisions about buying or selling tokens, products, services, or any other assets and should not be interpreted as such.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. CredShields does not provide any warranty or representation about the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business. The report is not intended to be used as investment advice and should not be relied upon as such.

CredShields Audit team is not responsible for any decisions or actions taken by any third party based on the report.

# Your Secure Future Starts Here

At CredShields, we're more than just auditors. We're your strategic partner in ensuring a secure Web3 future. Our commitment to your success extends beyond the report, offering ongoing support and guidance to protect your digital assets

SCS Advocates

OWASP Smart Contract Top 10 Security Risks & Vulnerabilities

20A TANJONG PAGAR ROAD, SINGAPORE (088443)

info@credshields.com